

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

UNIVERSIDAD TECNOLÓGICA NACIONAL

Bases de Datos II

Trabajo Práctico Integrador

Parte 1: Propuesta, Modelado y Persistencia Cloud

Dominio de negocio: E-Commerce de Tecnología

Motor de base de datos: MongoDB Atlas — Cluster M0 gratuito

Integrantes: Sussini Guanziroli Patricio
Vazquez Matias Ezequiel

Año: 2026

1. Descripción del Negocio

TechStore es una plataforma de comercio electrónico especializada en la venta de productos tecnológicos: notebooks, celulares, periféricos, accesorios y componentes de hardware. Los clientes registrados pueden explorar un catálogo organizado por categorías, agregar productos al carrito, realizar pedidos y hacer seguimiento de sus compras. Los administradores gestionan el inventario, las categorías y pueden aplicar bajas lógicas tanto a productos como a clientes.

Actores del sistema

- **Cliente:** se registra, navega el catálogo y realiza pedidos.
- **Administrador:** gestiona productos, categorías y clientes.
- **Sistema:** registra pedidos, actualiza stock y aplica bajas lógicas.

Operaciones principales

- Alta, consulta y baja lógica de clientes.
- Alta, modificación, consulta y baja lógica de productos.
- Gestión de categorías de productos.
- Registro de pedidos con ítems detallados.
- Consulta de historial de compras por cliente.

2. Diseño de Colecciones

El modelo se compone de cuatro colecciones principales. El dominio fue elegido porque presenta escenarios que justifican tanto el uso de Embedding como de Linking, y requiere baja lógica en múltiples entidades.

Colección	Descripción
clientes	Datos personales y de contacto de los compradores registrados.
categorias	Clasificación de los productos del catálogo.
productos	Ítems del catálogo con especificaciones técnicas, precio y stock.
pedidos	Órdenes de compra realizadas por los clientes.

3. Esquemas JSON de las Colecciones

3.1 — Colección: clientes

La dirección del cliente se embebe en el documento principal porque siempre se consulta junto con los datos del cliente (para la entrega de pedidos) y no tiene existencia ni acceso independiente.

```
{
  "_id": ObjectId("64a1f2e3b5c9d80012345678"),
  "nombre": "Lucía Fernández",
  "email": "lucia@email.com",
  "telefono": "11-4567-8901",
  "direccion": {
    "calle": "Av. Corrientes 1234",
    "ciudad": "Buenos Aires",
    "provincia": "CABA",
    "codigo_postal": "C1043"
  },
  "fecha_registro": ISODate("2024-01-15T10:30:00Z"),
  "activo": true
}
```

3.2 — Colección: categorías

Las categorías son entidades independientes referenciadas por los productos. Una categoría puede modificarse sin afectar a los productos. Se aplica Linking.

```
{
  "_id": ObjectId("64a1f2e3b5c9d80099887766"),
  "nombre": "Computadoras y Laptops",
  "descripcion": "Equipos de escritorio y portátiles",
  "slug": "computadoras-laptops",
  "activo": true
}
```

3.3 — Colección: productos

Las especificaciones técnicas se embeben porque son parte inherente del producto y siempre se muestran juntas. La categoría se referencia mediante `categoria_id` (Linking) porque es una entidad independiente con ciclo de vida propio.

```
{
  "_id": ObjectId("64a1f2e3b5c9d80087654321"),
  "nombre": "Notebook Lenovo IdeaPad 3",
  "descripcion": "Laptop 15.6 pulgadas, 8GB RAM, SSD 512GB",
  "categoria_id": ObjectId("64a1f2e3b5c9d80099887766"),
  "precio": 850000,
  "stock": 12,
  "imagenes": [
    "https://tienda.com/imgs/lenovo-front.jpg",
    "https://tienda.com/imgs/lenovo-side.jpg"
  ],
  "especificaciones": {
    "procesador": "Intel Core i5-1235U",
    "ram": "8GB DDR4",
    "almacenamiento": "512GB SSD NVMe",
    "pantalla": "15.6 FHD IPS"
  },
  "fecha_alta": ISODate("2024-01-10T08:00:00Z"),
  "activo": true
}
```

3.4 — Colección: pedidos

Los ítems del pedido se embeben dentro del pedido. Esto preserva el precio histórico: si el producto cambia de precio en el futuro, el pedido conserva el valor al momento de la compra. El cliente se referencia con `cliente_id` (Linking) porque tiene existencia independiente.

```
{
  "_id": ObjectId("64a1f2e3b5c9d80011223344"),
  "cliente_id": ObjectId("64a1f2e3b5c9d80012345678"),
  "fecha_pedido": ISODate("2024-03-20T14:25:00Z"),
  "estado": "entregado",
  "items": [
    {
      "producto_id": ObjectId("64a1f2e3b5c9d80087654321"),
      "nombre_producto": "Notebook Lenovo IdeaPad 3",
      "cantidad": 1,
      "precio_unitario": 850000
    },
    {
      "producto_id": ObjectId("64a1f2e3b5c9d80055443322"),
      "nombre_producto": "Mouse Inalámbrico Logitech M185",
      "cantidad": 2,
      "precio_unitario": 18500
    }
  ],
  "direccion_entrega": {
    "calle": "Av. Corrientes 1234",
    "ciudad": "Buenos Aires",
    "provincia": "CABA",
    "codigo_postal": "C1043"
  },
  "total": 887000,
  "activo": true
}
```

4. Justificaciones Técnicas

4.1 — Embedding (Documentos embebidos)

Se aplicó Embedding siguiendo el principio «data that is accessed together, should be stored together».

Atributo embebido	Justificación	Tipo de relación
dirección en clientes	Siempre se consulta con los datos del cliente. No tiene acceso ni existencia independiente.	1 a 1 estricta con el padre
especificaciones en productos	Son inseparables del producto. Se muestran en la ficha y nunca se acceden por separado.	Datos jerárquicos, lectura conjunta siempre
items en pedidos	Representan una instantánea histórica. Precio y nombre se persisten al momento de compra para no perder trazabilidad.	Inmutabilidad histórica del pedido
imagenes en productos	Array de URLs asociadas al producto. Se cargan siempre junto con el producto.	Lista de primitivos ligados al padre

4.2 — Linking (Referencias entre colecciones)

Se aplicó Linking cuando las entidades tienen vida propia, pueden actualizarse de forma independiente o la relación es débil:

Referencia	Justificación	Tipo de relación
categoria_id en productos	Una categoría puede modificarse, renombrarse o darse de baja sin afectar a los productos.	1 a N (1 categoría → N productos)
cliente_id en pedidos	El cliente existe independientemente. Sus datos pueden actualizarse sin modificar los pedidos.	1 a N (1 cliente → N pedidos)
producto_id en items de pedido	Referencia al producto original para trazabilidad. Precio y nombre se embeben por razones históricas (patrón híbrido).	Referencia + snapshot embebido

Patrón híbrido — items[] en pedidos

Dentro de cada ítem del pedido se combina embedding y linking, en donde se embebe nombre_producto y precio_unitario (snapshot histórico) y se referencia el producto_id. Esto garantiza integridad histórica del precio sin perder la trazabilidad hacia el catálogo original.

5. Implementación de Baja Lógica

No se eliminarán documentos físicamente de ninguna colección. La baja lógica se implementa mediante el campo activo (Boolean) presente en todas las colecciones. Esto preserva la integridad referencial, permitiendo evitar conflictos en casos como: un cliente dado de baja puede tener pedidos históricos; un producto discontinuado puede aparecer en compras anteriores.

Colección	Campo de baja lógica
clientes	activo: true / false
productos	activo: true / false
categorias	activo: true / false
pedidos	activo: true / false

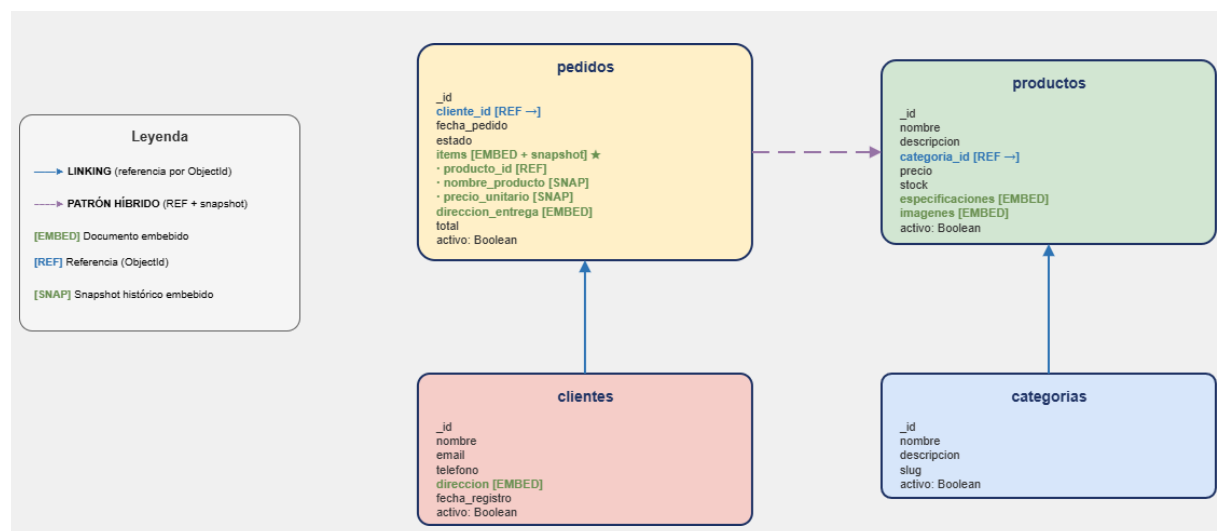
Comandos de baja lógica y consulta

```
// Dar de baja un producto:
db.productos.updateOne(
  { _id: ObjectId("64a1f2e3b5c9d80087654321") },
  { $set: { activo: false } }
)

// Dar de baja un cliente:
db.clientes.updateOne(
  { _id: ObjectId("64a1f2e3b5c9d80012345678") },
  { $set: { activo: false } }
)

// Consultar solo registros activos:
db.productos.find({ activo: true })
```

6. Mapa de Relaciones entre Colecciones



7. Infraestructura Cloud — MongoDB Atlas

7.1 — Configuración del Cluster M0

- Nombre del cluster: cluster-techstore

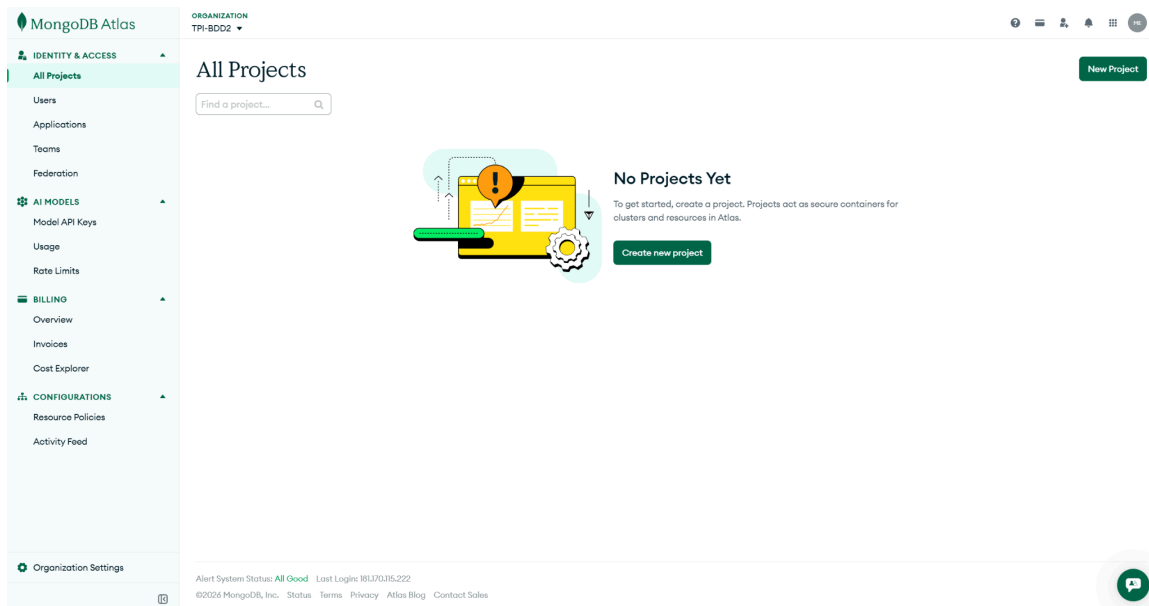


Figura 1 — Cluster M0 creado en MongoDB Atlas

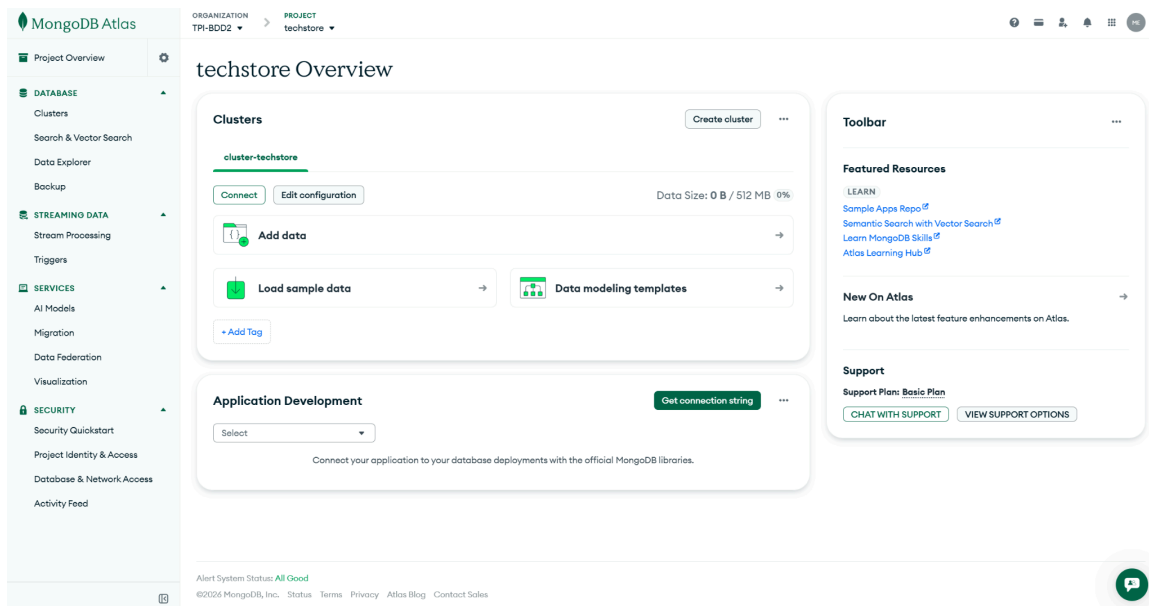


Figura 2 — Vista general del Cluster M0

7.2 — Database Access

- **Usuarios:** db_admin & db_user
- **Permisos:** Admin & Read any database

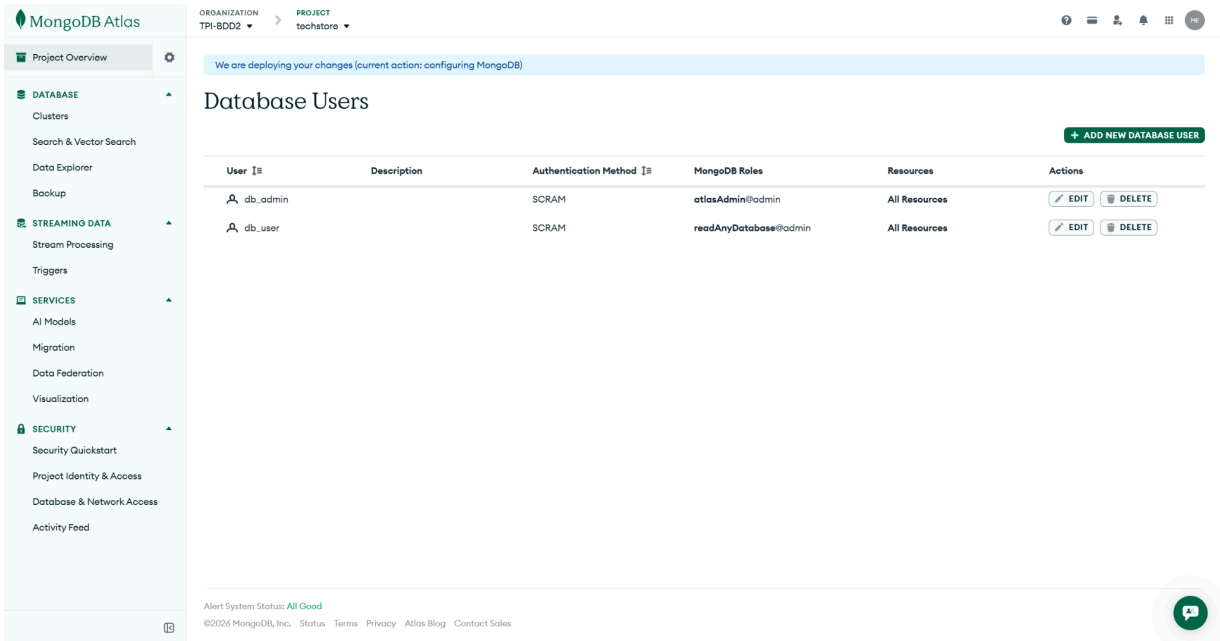


Figura 3 — Usuarios configurados en Database Access

7.3 — Network Access

- Se habilita acceso desde cualquier IP: 0.0.0.0/0

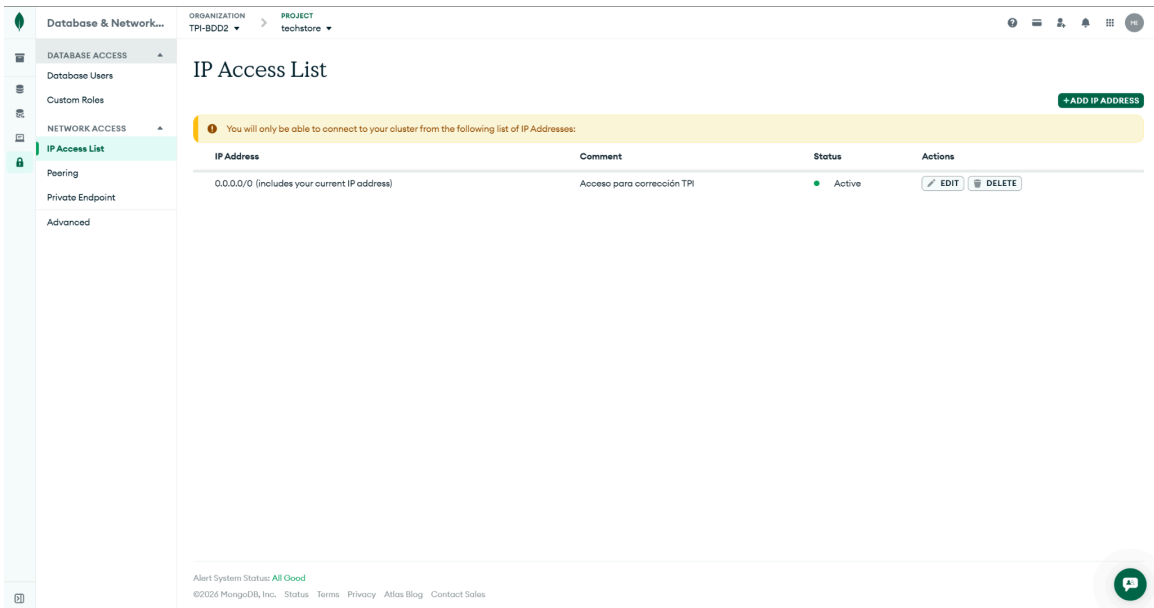


Figura 4 — Regla de Network Access (0.0.0.0/0)

7.4 — Base de datos y colecciones

- **Base de datos:** techstore_db
- **Colecciones:** clientes, categorias, productos, pedidos

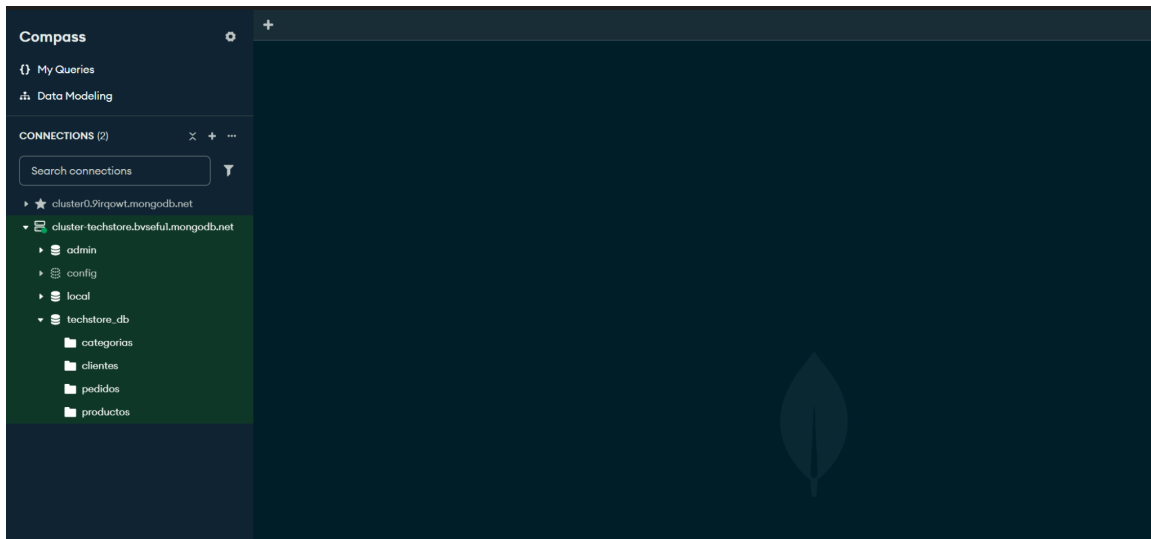


Figura 5 — Colecciones creadas en Atlas

7.5 — Connection String

```
// Compass
mongodb+srv://db_user:db_user_2468@cluster-techstore.bvseful.mongodb.net/

// Shell
mongosh "mongodb+srv://cluster-techstore.bvseful.mongodb.net/" --apiVersion 1 --username db_user

// User: db_user
// Pw: db_user_2468
```

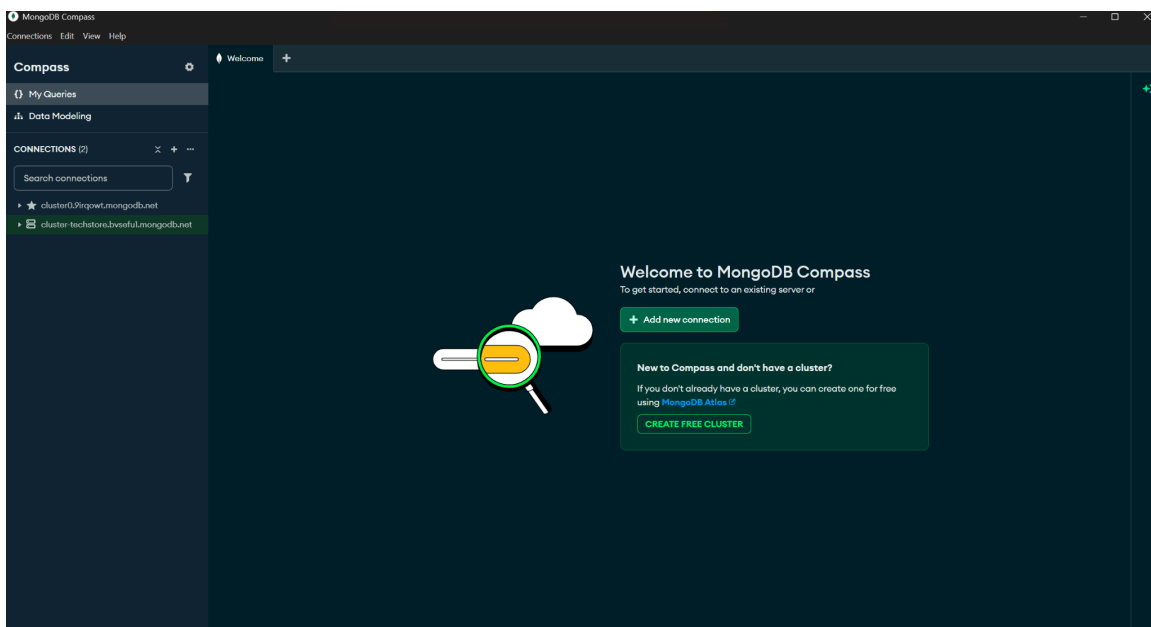


Figura 6 — Conexión al Cluster desde MongoDB Compass

8. Comandos para el Armado de la Base de Datos

Los siguientes comandos se utilizaron en con mongosh a través de Compass.

Crear la base de datos

```
use techstore_db
```

Crear colecciones con validación de esquema

```
// ----- CLIENTES -----
db.createCollection("clientes", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nombre", "email", "direccion", "fecha_registro", "activo"],
      properties: {
        nombre: { bsonType: "string" },
        email: { bsonType: "string" },
        telefono: { bsonType: "string" },
        direccion: {
          bsonType: "object",
          required: ["calle", "ciudad", "provincia", "codigo_postal"],
          properties: {
            calle: { bsonType: "string" },
            ciudad: { bsonType: "string" },
            provincia: { bsonType: "string" },
            codigo_postal: { bsonType: "string" }
          }
        },
        fecha_registro: { bsonType: "date" },
        activo: { bsonType: "bool" }
      }
    }
  }
})

// ----- CATEGORIAS -----
db.createCollection("categorias", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nombre", "activo"],
      properties: {
        nombre: { bsonType: "string" },
        descripcion: { bsonType: "string" },
        slug: { bsonType: "string" },
        activo: { bsonType: "bool" }
      }
    }
  }
})

// ----- PRODUCTOS -----
db.createCollection("productos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nombre", "categoria_id", "precio", "stock", "activo"],
      properties: {
        nombre: { bsonType: "string" },
        descripcion: { bsonType: "string" },
        categoria_id: { bsonType: "objectId" },
        precio: { bsonType: "number" },
        stock: { bsonType: "int" },
        imagenes: { bsonType: "array", items: { bsonType: "string" } },
        especificaciones: { bsonType: "object" },
      }
    }
  }
})
```

```

        fecha_alta: { bsonType: "date" },
        activo:     { bsonType: "bool" }
    }
}
})

// ----- PEDIDOS -----
db.createCollection("pedidos", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["cliente_id", "fecha_pedido", "estado", "items", "total", "activo"],
      properties: {
        cliente_id: { bsonType: "objectId" },
        fecha_pedido: { bsonType: "date" },
        estado: {
          bsonType: "string",
          enum: ["pendiente", "confirmado", "enviado", "entregado", "cancelado"]
        },
        items: {
          bsonType: "array",
          items: {
            bsonType: "object",
            required: ["producto_id", "nombre_producto", "cantidad", "precio_unitario"],
            properties: {
              producto_id: { bsonType: "objectId" },
              nombre_producto: { bsonType: "string" },
              cantidad: { bsonType: "int" },
              precio_unitario: { bsonType: "number" }
            }
          }
        },
        direccion_entrega: { bsonType: "object" },
        total: { bsonType: "number" },
        activo: { bsonType: "bool" }
      }
    }
  }
})

```

Ejemplo de inserción de documentos

```
// 1. Insertar categoría
const catId = new ObjectId()
db.categorias.insertOne({
  _id: catId,
  nombre: "Computadoras y Laptops",
  descripcion: "Equipos de escritorio y portátiles",
  slug: "computadoras-laptops",
  activo: true
})

// 2. Insertar producto (referencia a catId)
const prodId = new ObjectId()
db.productos.insertOne({
  _id: prodId,
  nombre: "Notebook Lenovo IdeaPad 3",
  descripcion: "Laptop 15.6 pulgadas, 8GB RAM, SSD 512GB",
  categoria_id: catId,
  precio: 850000,
  stock: 12,
  imagenes: [
    "https://tienda.com/imgs/lenovo-front.jpg",
    "https://tienda.com/imgs/lenovo-side.jpg"
  ],
  especificaciones: {
    procesador: "Intel Core i5-1235U",
    ram: "8GB DDR4",
    almacenamiento: "512GB SSD NVMe",
    pantalla: "15.6 FHD IPS"
  },
  fecha_alta: new Date("2024-01-10"),
  activo: true
})

// 3. Insertar cliente
const cliId = new ObjectId()
db.clientes.insertOne({
  _id: cliId,
  nombre: "Lucía Fernández",
  email: "lucia@email.com",
  telefono: "11-4567-8901",
  direccion: {
    calle: "Av. Corrientes 1234",
    ciudad: "Buenos Aires",
    provincia: "CABA",
    codigo_postal: "C1043"
  },
  fecha_registro: new Date("2024-01-15"),
  activo: true
})

// 4. Insertar pedido (snapshot de precio embebido)
db.pedidos.insertOne({
  cliente_id: cliId,
  fecha_pedido: new Date("2024-03-20"),
  estado: "entregado",
  items: [
    {
      producto_id: prodId,
      nombre_producto: "Notebook Lenovo IdeaPad 3",
      cantidad: 1,
      precio_unitario: 850000
    }
  ],
  direccion_entrega: {
    calle: "Av. Corrientes 1234",
    ciudad: "Buenos Aires",
    provincia: "CABA",
    codigo_postal: "C1043"
  },
})
```

```
total: 850000,  
activo: true  
})
```

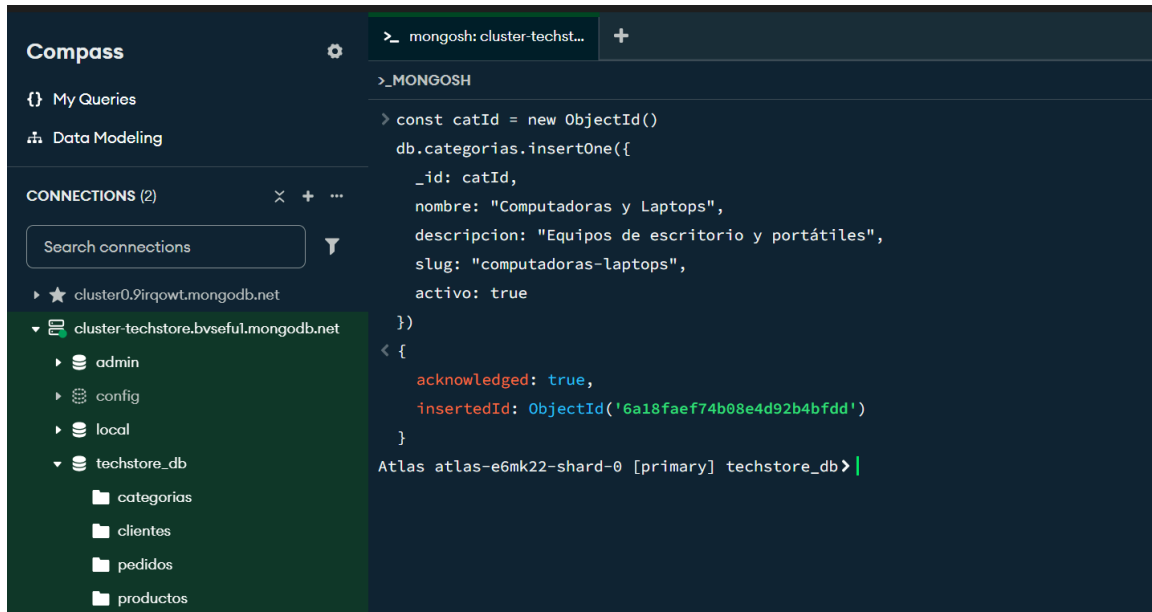


Figura 7 — Inserts en MongoDB Compass (1)

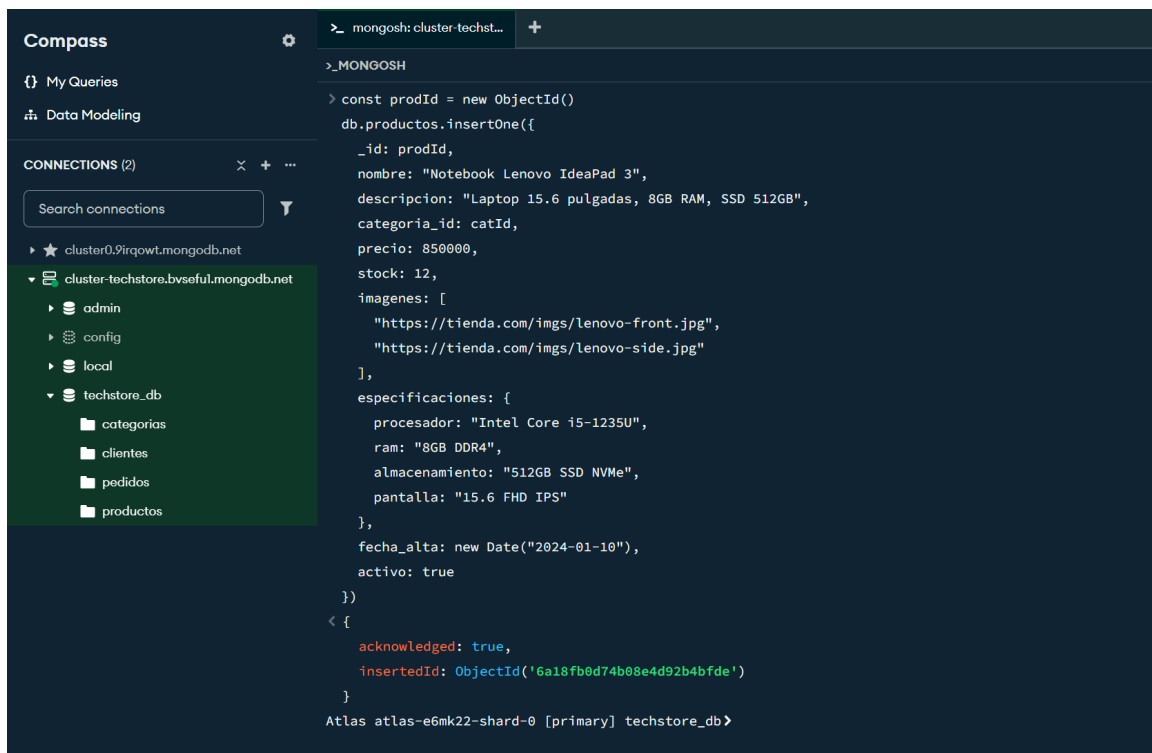


Figura 8 — Inserts en MongoDB Compass (2)

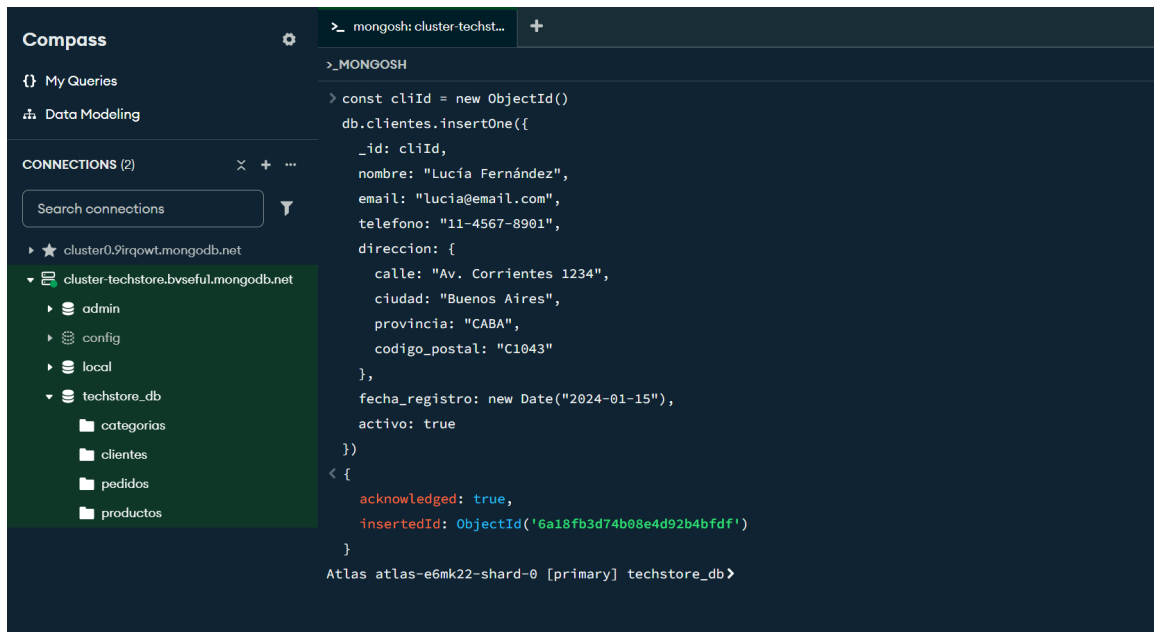


Figura 9 — Colecciones con documentos insertados (1)

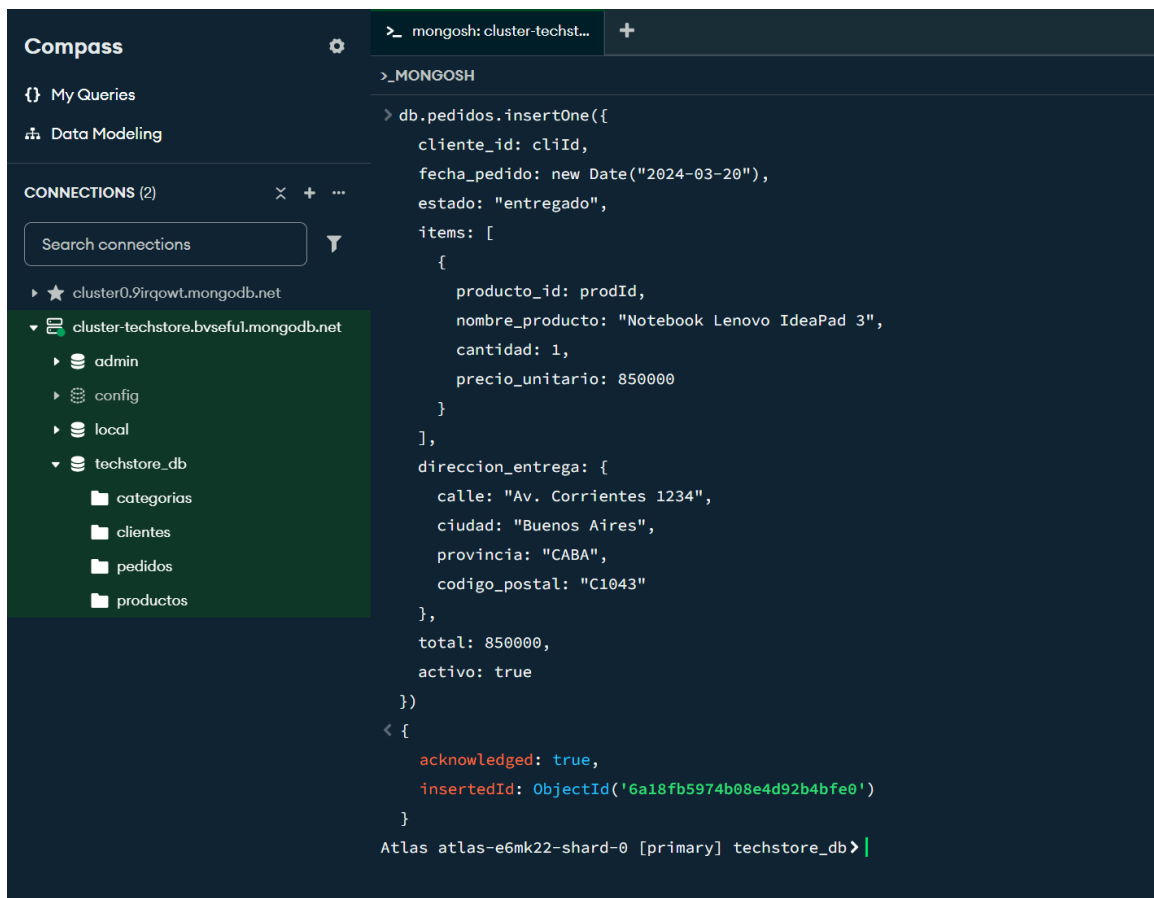


Figura 10 — Colecciones con documentos insertados (2)

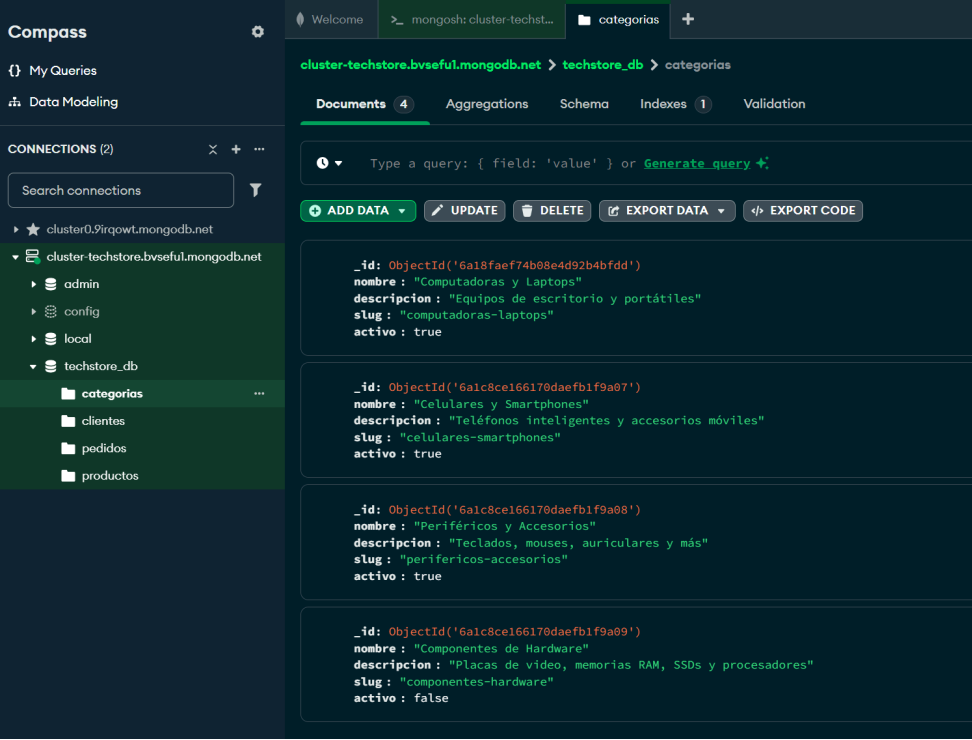


Figura 11 — Documentos en colección categorias

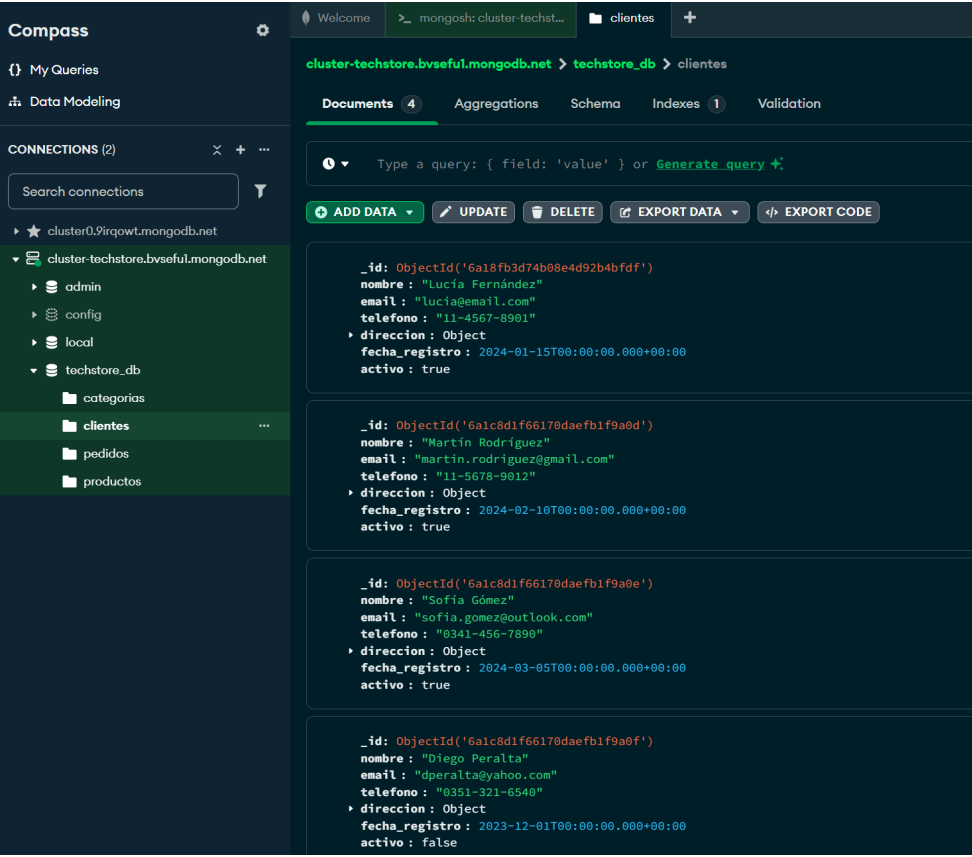


Figura 12 — Documentos en colección clientes

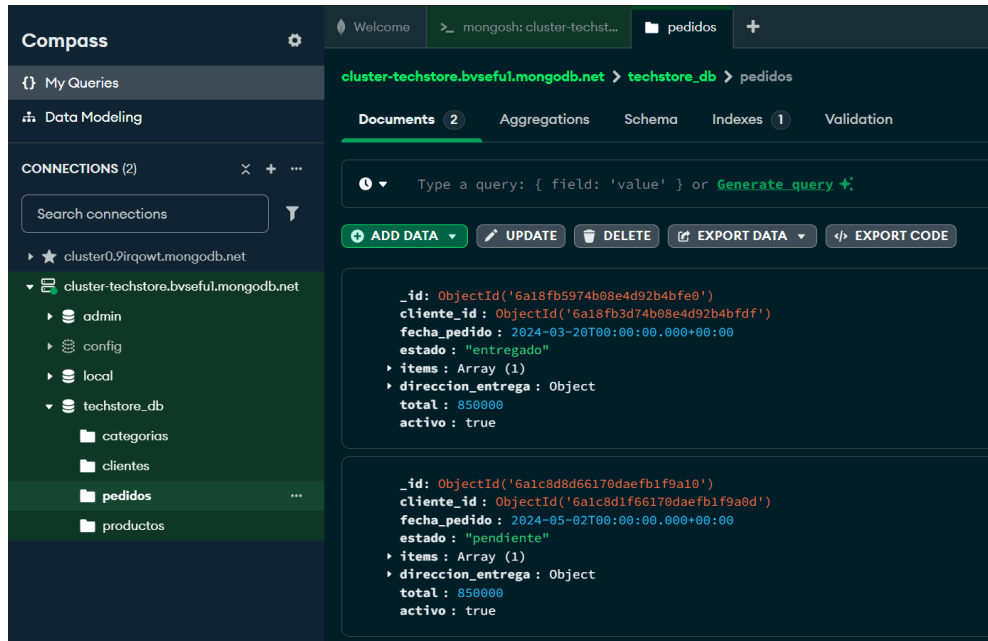


Figura 13 — Documentos en colección pedidos

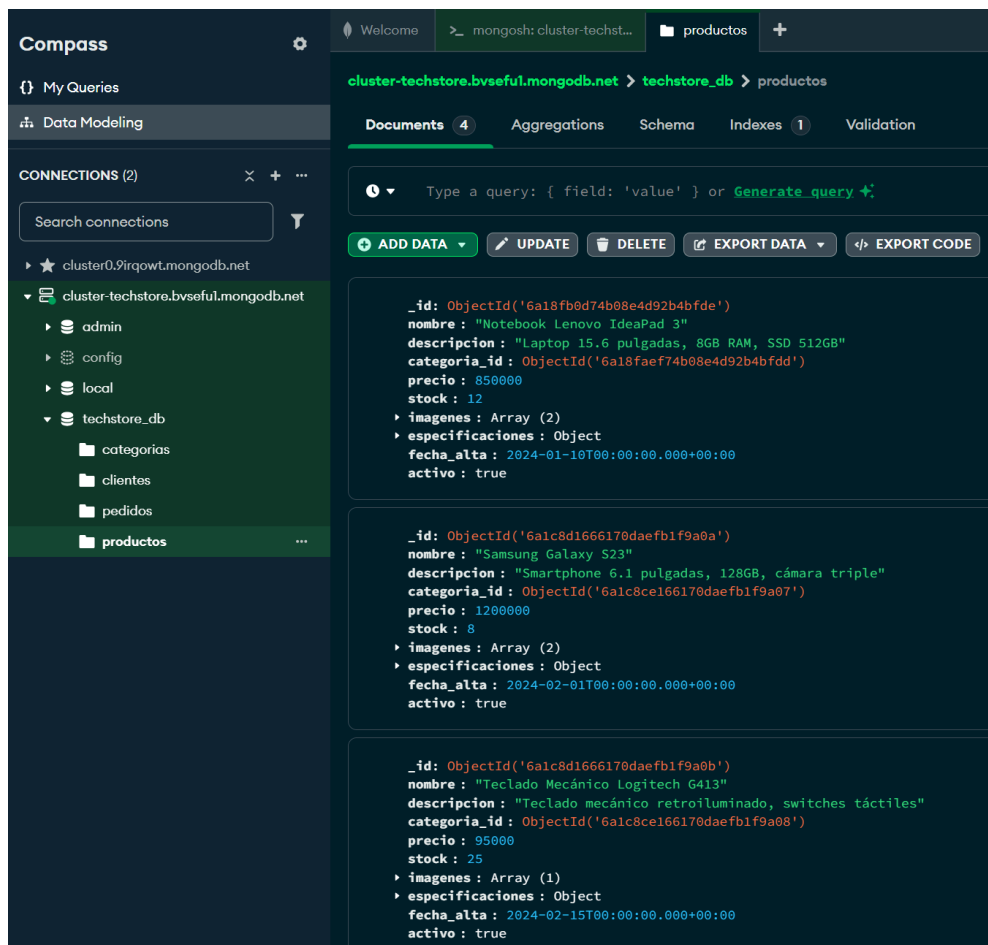


Figura 14 — Documentos en colección productos